

```

import logging
import sqlite3
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup, ReplyKey
from telegram.ext import (
    ApplicationBuilder,
    CommandHandler,
    ContextTypes,
    CallbackQueryHandler,
    MessageHandler,
    filters,
    ConversationHandler
)

# Configuration du logging
logging.basicConfig(
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    level=logging.INFO
)

TOKEN = '8523130769:AAHaq2-P9pQhb6okb36vmvehi0YKYsoYp2c'
ADMIN_IDS = [8474448647]

# États de la conversation
PHONE, ADDRESS, ORDER_DETAILS = range(3)

# --- Catalogue des produits ---
PRODUCTS = {
    'mousseux': {'name': '🍷 Mousseux', 'price': 50},
    'filtre': {'name': '☕ Filtré', 'price': 100},
}

# --- Fonctions de Base de Données ---

def get_db_connection():
    conn = sqlite3.connect('bot_database.db')
    conn.row_factory = sqlite3.Row
    return conn

def register_user(telegram_id, first_name, last_name, username):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('''
        INSERT OR IGNORE INTO users (telegram_id, first_name, last_name, username
        VALUES (?, ?, ?, ?)
    ''')

```

```

        '', (telegram_id, first_name, last_name, username))
    conn.commit()
    conn.close()

def update_user_info(telegram_id, phone=None):
    conn = get_db_connection()
    cursor = conn.cursor()
    if phone:
        cursor.execute('UPDATE users SET phone_number = ? WHERE telegram_id = ?',
        conn.commit()
    conn.close()

def get_user_id(telegram_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('SELECT id FROM users WHERE telegram_id = ?', (telegram_id,))
    row = cursor.fetchone()
    conn.close()
    return row['id'] if row else None

def get_user_points(telegram_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('SELECT loyalty_points FROM users WHERE telegram_id = ?', (tel
    row = cursor.fetchone()
    conn.close()
    return row['loyalty_points'] if row else 0

def save_simple_order(telegram_id, phone, address, details, total):
    conn = get_db_connection()
    cursor = conn.cursor()
    user_id = get_user_id(telegram_id)

    cursor.execute('''
        INSERT INTO orders (user_id, total_amount, status)
        VALUES (?, ?, ?)
    ''', (user_id, total, 'pending'))
    order_id = cursor.lastrowid

    cursor.execute('UPDATE users SET loyalty_points = loyalty_points + 1 WHERE id

    conn.commit()
    conn.close()
    return order_id

# --- Helpers Panier ---

```

```

def get_cart(context) -> dict:
    """Retourne le panier actuel (dict product_key -> qty)."""
    return context.user_data.setdefault('cart', {})

def cart_total(cart: dict) -> int:
    return sum(PRODUCTS[k]['price'] * qty for k, qty in cart.items())

def cart_summary(cart: dict) -> str:
    if not cart:
        return "🛒 Votre panier est vide."
    lines = []
    for key, qty in cart.items():
        p = PRODUCTS[key]
        lines.append(f"• {p['name']} x{qty} = {p['price'] * qty} €")
    lines.append(f"\n💰 **Total : {cart_total(cart)} €**")
    return "\n".join(lines)

def catalog_keyboard(cart: dict) -> InlineKeyboardMarkup:
    """Clavier du catalogue avec boutons + / - et total."""
    buttons = []
    for key, product in PRODUCTS.items():
        qty = cart.get(key, 0)
        qty_label = f" ({qty})" if qty else ""
        buttons.append([
            InlineKeyboardButton(f"➖", callback_data=f'cart_remove_{key}'),
            InlineKeyboardButton(f"{product['name']} - {product['price']} €{qty_label}"),
            InlineKeyboardButton(f"➕", callback_data=f'cart_add_{key}'),
        ])

    total = cart_total(cart)
    if total > 0:
        buttons.append([
            InlineKeyboardButton(f"🛒 Voir le panier ({total} €)", callback_data=f'cart_summary'),
            InlineKeyboardButton("🏠 Menu Principal", callback_data='back')
        ])
    return InlineKeyboardMarkup(buttons)

# --- Handlers du Bot ---

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user = update.effective_user
    register_user(user.id, user.first_name, user.last_name, user.username)
    context.user_data.clear()

    keyboard = [
        [
            InlineKeyboardButton("📦 Catalogue & Commander", callback_data='show_cat'),
            InlineKeyboardButton("📇 Ma Carte de Fidélité", callback_data='loyalty_c'),
            InlineKeyboardButton("❓ Aide", callback_data='help')
        ]
    ]

```

```

        await update.message.reply_text(
            f"Bonjour {user.first_name} ! Bienvenue sur notre bot de commande.\n"
            "Prêt à passer votre commande ?",
            reply_markup=InlineKeyboardMarkup(keyboard)
        )

async def show_catalog(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    cart = get_cart(context)
    await query.edit_message_text(
        "🛒 *Catalogue*\n\nChoisissez vos produits en cliquant sur + / - :",
        parse_mode='Markdown',
        reply_markup=catalog_keyboard(cart)
    )

async def cart_add(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    key = query.data.replace('cart_add_', '')
    cart = get_cart(context)
    cart[key] = cart.get(key, 0) + 1
    await query.edit_message_text(
        "🛒 *Catalogue*\n\nChoisissez vos produits en cliquant sur + / - :",
        parse_mode='Markdown',
        reply_markup=catalog_keyboard(cart)
    )

async def cart_remove(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    key = query.data.replace('cart_remove_', '')
    cart = get_cart(context)
    if cart.get(key, 0) > 0:
        cart[key] -= 1
        if cart[key] == 0:
            del cart[key]
    await query.edit_message_text(
        "🛒 *Catalogue*\n\nChoisissez vos produits en cliquant sur + / - :",
        parse_mode='Markdown',
        reply_markup=catalog_keyboard(cart)
    )

async def view_cart(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    cart = get_cart(context)

```

```

summary = cart_summary(cart)

buttons = []
if cart:
    buttons.append([InlineKeyboardButton("✅ Confirmer et commander", callback_data='confirm_order')])
    buttons.append([InlineKeyboardButton("⬅️ Retour au catalogue", callback_data='catalog')])
    buttons.append([InlineKeyboardButton("🗑️ Vider le panier", callback_data='clear_cart')])

await query.edit_message_text(
    f"🛒 *Votre Panier*\n\n{summary}",
    parse_mode='Markdown',
    reply_markup=InlineKeyboardMarkup(buttons)
)

async def clear_cart(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer("Panier vidé.")
    context.user_data['cart'] = {}
    await query.edit_message_text(
        f"🛍️ *Catalogue*\n\nChoisissez vos produits en cliquant sur + / - :",
        parse_mode='Markdown',
        reply_markup=catalog_keyboard({})
    )

# --- Conversation de commande (phone → address → done) ---

async def start_order_callback(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Déclenché par 'confirm_order' — démarre la conversation."""
    query = update.callback_query
    await query.answer()

    cart = get_cart(context)
    if not cart:
        await query.answer("Votre panier est vide !", show_alert=True)
        return ConversationHandler.END

    contact_keyboard = [[KeyboardButton("📞 Partager mon numéro", request_contact=True)]]
    reply_markup = ReplyKeyboardMarkup(contact_keyboard, one_time_keyboard=True, resize_keyboard=False)

    await query.message.reply_text(
        "Super ! Pour finaliser la commande, veuillez nous transmettre votre numéro de téléphone."
    )
    return PHONE

async def get_phone(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if update.message.contact:

```

```

        phone = update.message.contact.phone_number
    else:
        phone = update.message.text

    context.user_data['order_phone'] = phone
    update_user_info(update.effective_user.id, phone)

    await update.message.reply_text(
        "Parfait ! Quelle est votre adresse de livraison ?",
        reply_markup=None
    )
    return ADDRESS

async def get_address(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data['order_address'] = update.message.text

    cart = get_cart(context)
    summary = cart_summary(cart)

    await update.message.reply_text(
        f"📦 Récapitulatif de votre commande :\n\n{summary}\n\n"
        "Ajoutez une note ou des instructions spéciales (ou tapez 'aucune') :"
    )
    return ORDER_DETAILS

async def get_order_details(update: Update, context: ContextTypes.DEFAULT_TYPE):
    details = update.message.text
    user = update.effective_user
    phone = context.user_data.get('order_phone')
    address = context.user_data.get('order_address')
    cart = get_cart(context)
    total = cart_total(cart)
    summary = cart_summary(cart)

    order_id = save_simple_order(user.id, phone, address, details, total)

    # Notification Admin
    admin_msg = (
        f"🔴 NOUVELLE COMMANDE ! (ID: #{order_id})\n\n"
        f"👤 Client : {user.first_name} (@{user.username})\n\n"
        f"☎️ Tél : {phone}\n\n"
        f"📍 Adresse : {address}\n\n"
        f"🛒 Produits :\n\n{summary}\n\n"
        f"📝 Note : {details}"
    )
    for admin_id in ADMIN_IDS:
        try:

```

```

        await context.bot.send_message(chat_id=admin_id, text=admin_msg)
    except Exception as e:
        logging.error(f"Erreur envoi admin {admin_id}: {e}")

# Confirmation client
confirm_msg = (
    f"✅ Commande confirmée ! (ID: #{order_id})\n\n"
    f"{summary}\n\n"
    f"📍 Livraison : {address}\n\n"
    f"Nous reviendrons vers vous rapidement. Merci ! 🎁 +1 point de fidélité"
)

context.user_data['cart'] = {}

keyboard = [[InlineKeyboardButton("⬅️ Menu Principal", callback_data='back_to_menu')]]
await update.message.reply_text(confirm_msg, reply_markup=InlineKeyboardMarkup(keyboard))
return ConversationHandler.END

async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Commande annulée.", reply_markup=None)
    return ConversationHandler.END

# --- Handler boutons généraux ---

async def button_handler(update: Update, context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()

    if query.data == 'loyalty_card':
        points = get_user_points(query.from_user.id)
        await query.edit_message_text(
            f"📜 *Votre Carte de Fidélité*\n\n"
            f"Solde actuel : {points} point(s)\n\n"
            f"Chaque commande vous rapporte 1 point !",
            parse_mode='Markdown',
            reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton("⬅️ Retour",
            )
        elif query.data == 'help':
            await query.edit_message_text(
                f"❓ *Aide*\n\n"
                f"• Cliquez sur 'Catalogue & Commander' pour voir nos produits.\n"
                f"• Ajoutez des articles avec + et retirez avec -.\n"
                f"• Confirmez votre panier et suivez les étapes.\n"
                f"• Cumulez des points à chaque achat !",
                parse_mode='Markdown',
                reply_markup=InlineKeyboardMarkup([[InlineKeyboardButton("⬅️ Retour",
            )

```

```

elif query.data == 'back_to_main':
    keyboard = [
        [InlineKeyboardButton("🛍️ Catalogue & Commander", callback_data='show
        [InlineKeyboardButton("📇 Ma Carte de Fidélité", callback_data='loyal
        [InlineKeyboardButton("❓ Aide", callback_data='help')]
    ]
    await query.edit_message_text(
        "Que souhaitez-vous faire aujourd'hui ?",
        reply_markup=InlineKeyboardMarkup(keyboard)
    )

# --- Lancement du Bot ---

if __name__ == '__main__':
    application = ApplicationBuilder().token(TOKEN).build()

    conv_handler = ConversationHandler(
        entry_points=[CallbackQueryHandler(start_order_callback, pattern='^confir
        states={
            PHONE: [MessageHandler(filters.CONTACT | filters.TEXT & ~filt
            ADDRESS: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_a
            ORDER_DETAILS: [MessageHandler(filters.TEXT & ~filters.COMMAND, get_o
        },
        fallbacks=[CommandHandler('cancel', cancel)],
    )

    application.add_handler(CommandHandler('start', start))
    application.add_handler(conv_handler)

# Handlers catalogue et panier
application.add_handler(CallbackQueryHandler(show_catalog, pattern='^show_ca
application.add_handler(CallbackQueryHandler(cart_add, pattern='^cart_ad
application.add_handler(CallbackQueryHandler(cart_remove, pattern='^cart_re
application.add_handler(CallbackQueryHandler(view_cart, pattern='^view_ca
application.add_handler(CallbackQueryHandler(clear_cart, pattern='^clear_c

# Handler boutons généraux (loyalty, help, back)
application.add_handler(CallbackQueryHandler(button_handler))

print("Bot avec catalogue et panier en cours d'exécution...")
application.run_polling()

```